

Travel Insurance Claim Prediction

A Classification Study Using Decision Tree, Logistic Regression, and Neural Network Models

Introduction

This project builds predictive models to determine whether a travel insurance policy will result in a claim. Using a dataset of past insurance sales and outcomes, we apply three supervised learning methods (Decision Tree, Logistic Regression, and a Neural Network) to solve a binary classification problem, predicting the categorical outcome “Claim” (Yes/No) from ten input features. The goal is to help insurance providers identify high-risk policies; accurate prediction of claims supports premium-setting and risk management. Because the target variable is a discrete label, this is treated throughout as a classification task, evaluated primarily on precision, recall, and AUC rather than accuracy alone, for reasons discussed below.

Dataset Description

The raw dataset contains 63,326 records and 10 input features plus the target. Features include categorical fields (Agency, Agency Type, Distribution Channel, Product Name, Destination, Gender) and numeric fields (Duration, Net Sales, Commission, Age). The target column, Claim, takes two values (“Yes” or “No”), confirming a classification problem.

The dataset is severely imbalanced: only 1.46% of policies resulted in a claim. This single fact drives most of the modeling decisions described in this report: a model that simply predicts “No” for every policy would already score above 98% accuracy while being completely useless, so accuracy cannot be trusted as the primary metric anywhere in this project.

Exploratory Data Analysis

A correlation heatmap of the numeric features (Figure 1, left) shows that Net Sales and Commission are strongly positively correlated, as expected since higher sales typically yield higher commission. Every other numeric feature has weak mutual correlation, and, more importantly, every numeric feature has very low linear correlation with the Claim label itself. No single numeric feature linearly predicts claims, which suggests the models will need to rely on non-linear interactions and the categorical features (Agency, Product Name, Destination) rather than the numeric fields alone.

The class distribution bar chart (Figure 1, right) makes the imbalance visually unmistakable: the “No” bar dwarfs the “Yes” bar.



Figure 1. Correlation heatmap of numeric features (left) and class distribution of the target (right).

Closer inspection of the raw data also surfaced several data-quality issues that a straightforward `describe()` summary does not make obvious:

- **Duration:** 5 records have a negative value (as low as -2 days), which is not physically possible for a travel length.
- **Age:** 984 records have `Age = 118`, an isolated spike far outside the otherwise natural age distribution, a strong signal that 118 is a system placeholder for missing demographic data rather than a real age. (All 5 negative-Duration records also carry this `Age = 118` placeholder, so the two issues largely overlap.)
- **Net Sales:** 678 records are negative, most plausibly representing cancellations or refunds rather than data errors.
- **Gender:** missing on 45,107 of 63,326 records (about 71% of the data).
- **Agency, Product Name, and Destination** are high-cardinality categorical fields (16, 26, and 149 unique values respectively), with many Destination categories appearing only once in the entire dataset.

Dataset Pre-processing

Preprocessing addressed each of the issues identified in the EDA above:

- **Gender:** dropped as a column, rather than dropping every row missing it. Dropping rows instead of the column would have discarded 71% of the dataset for no benefit, since the column itself was the problem, not the other 10 fields on those rows.
- **Invalid Duration and Age:** the 5 rows with negative Duration and the 984 rows with the `Age = 118` placeholder were removed, since these values are extremely small in number relative to the $\sim 9.9\%$ share and would otherwise mislead any model relying on Duration or Age. This left 62,342 records.
- **Negative Net Sales:** rather than dropping or clipping these rows, a binary `Is_Refund` flag was added, preserving the information (that a cancellation/refund occurred) as an explicit signal instead of discarding it.
- **Target and categorical encoding:** Claim was encoded as binary (`1 = Yes`, `0 = No`). All five categorical predictors, including Agency, Product Name, and Destination, were one-hot encoded so that none of them are discarded before modeling.
- **Numeric scaling:** Duration, Net Sales, Commission, Age, and the `Is_Refund` flag were standardized (mean 0, unit variance), which matters most for the gradient-based Logistic Regression and Neural Network models.

One methodological detail is worth calling out explicitly: encoding and scaling were fit only on the training set, after the train/test split (see next section), using a scikit-learn `ColumnTransformer` combining a `StandardScaler` for numeric columns and a `OneHotEncoder(handle_unknown='ignore')` for categorical columns. Fitting the encoder on the full dataset before splitting would mean the transformation already “knows” about every category present in the test set, including the many Destination categories that occur only once. Fitting only on the training data is both the statistically correct order of operations and a more realistic simulation of production, where a genuinely new destination category would need to be handled gracefully rather than crash the pipeline. In practice, a small number of Destination categories present only in the test split are safely encoded as all-zero rather than causing an error.

Dataset Splitting

The data was split into training and test sets using a 70/30 ratio, with stratified sampling on the target label so that both sets preserve the original 1.47% claim rate. Given the extreme imbalance, stratification prevents a scenario where the rare “Yes” class is nearly absent from one of the two splits, which would make evaluation unreliable. This split was performed before any encoding or scaling was fit, for the reasons described above. The resulting training set contains 43,639 records and the test set 18,703 records, each expanding to 184 features after one-hot encoding.

Model Training & Testing

1. Decision Tree Classifier

A Decision Tree was trained with `class_weight='balanced'`, which upweights the rare “Yes” class during training so the tree is not simply optimizing for the majority class. Depth and leaf size were also constrained (`max_depth=10`, `min_samples_leaf=20`): an earlier, unconstrained version of this tree grew to depth 44 with 2,362 leaves and reached 99.8% training accuracy against only 97.0% test accuracy, a textbook sign that it was memorizing individual training rows rather than learning generalizable patterns. With the constraints in place, training accuracy (72.4%) and test accuracy (71.5%) are now close to each other, indicating the tree generalizes rather than overfits.

- **Accuracy:** 71.5% on the test set. On its own this number looks worse than before, but it is now a meaningful accuracy rather than one inflated by predicting the majority class almost exclusively.
- **Classification Report:** recall for the minority “Yes” class is 0.71 and precision is 0.04; for the majority “No” class, precision is 0.99 and recall is 0.72.
- **Confusion Matrix:** shown in Figure 2 (left panel). The tree now correctly flags 196 of 275 actual claims, at the cost of a large number of false positives among the “No” policies, an expected trade-off once a rare class is upweighted.
- **ROC and AUC:** AUC is 0.735, a substantial improvement over the 0.541 obtained by the earlier unconstrained tree, confirming that regularization did not just change the accuracy trade-off but genuinely improved the model's ability to rank claims above non-claims.

2. Logistic Regression

A Logistic Regression model was trained on the same scaled, encoded features, also with `class_weight='balanced'` so that it receives the same imbalance treatment as the Decision Tree.

- **Accuracy:** 79.7% on the test set.
- **Classification Report:** recall for “Yes” is 0.71 and precision is 0.05; for “No”, precision is 0.99 and recall is 0.80. Unlike an earlier version of this model (trained without class weighting), Logistic Regression here no longer collapses to predicting “No” for every record.
- **Confusion Matrix:** shown in Figure 2 (middle panel): 195 of 275 actual claims are correctly identified.
- **ROC and AUC:** AUC is 0.818, the best-ranked of the three models on this run.

3. Neural Network

A feedforward neural network (three hidden layers of 64, 32, and 16 units, ReLU activations, sigmoid output) was trained on the same inputs using Keras. It received an equivalent class-weighting scheme to the other two models (computed directly from the training label counts), and training used up to 50 epochs with early stopping on validation AUC (patience of 5 epochs, best weights restored) rather than a fixed, arbitrarily short epoch count. Training stopped after 9 epochs once validation AUC stopped improving.

- **Accuracy:** 77.6% on the test set.
- **Classification Report:** recall for “Yes” is 0.74 and precision is 0.05; for “No”, precision is 0.99 and recall is 0.78.
- **Confusion Matrix:** shown in Figure 2 (right panel).
- **ROC and AUC:** AUC is 0.824, the highest of the three models, consistent with the network's greater capacity to model non-linear feature interactions.

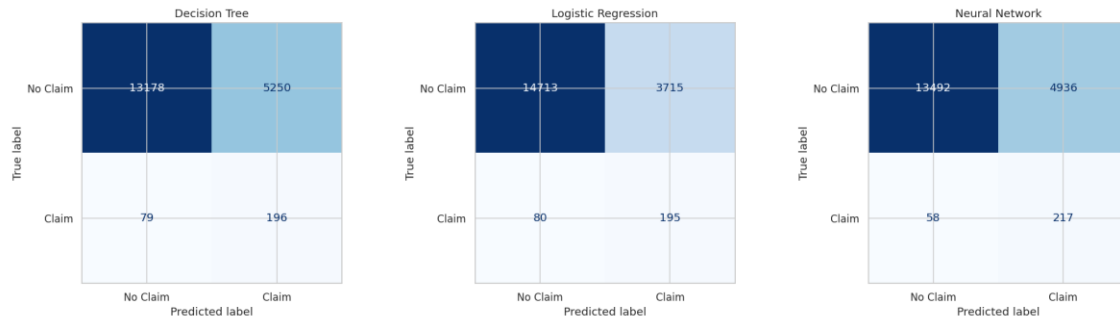


Figure 2. Confusion matrices for the Decision Tree, Logistic Regression, and Neural Network models on the test set.

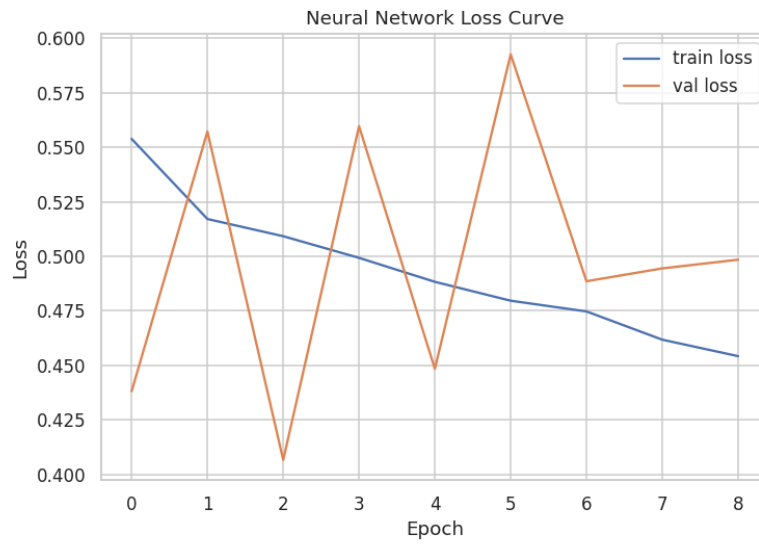


Figure 3. Neural network training and validation loss by epoch, showing early stopping at a sensible point rather than continued training past the point of improvement.

Model Selection and Comparison

Table 1 and Figure 4 summarize accuracy and AUC across the three models on the test set.

Model	Accuracy	Recall (Yes)	AUC
Decision Tree	0.715	0.71	0.735
Logistic Regression	0.797	0.71	0.818
Neural Network	0.776	0.74	0.824

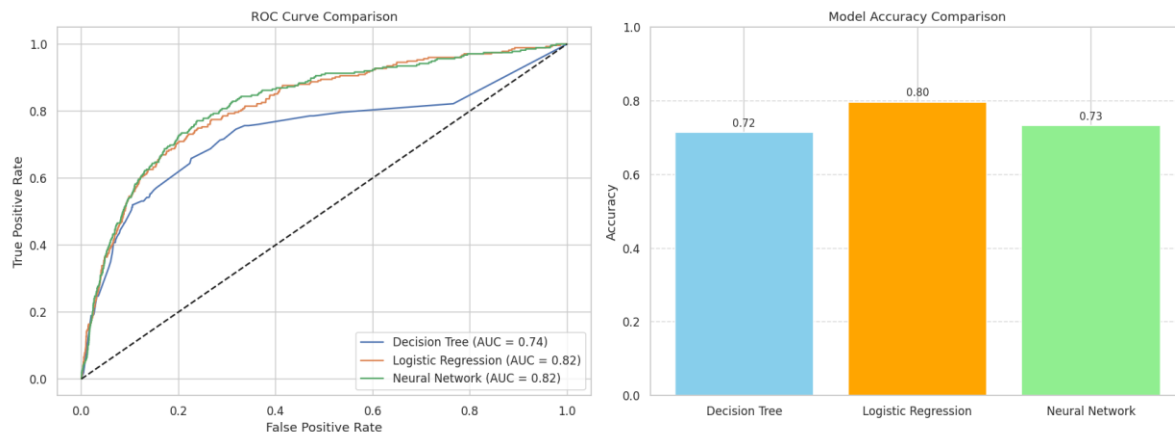


Figure 4. ROC curve comparison (left) and test accuracy comparison (right) across all three models.

All three ROC curves now sit well above the diagonal random-guessing line, including the Decision Tree, whose curve previously tracked the diagonal almost exactly. Accuracy is lowest for the Decision Tree (71.5%) and highest for Logistic Regression (79.7%), but, as emphasized throughout this report, accuracy is the least informative of these numbers given the class imbalance. Ranked by AUC, the Neural Network is marginally ahead (0.824), followed closely by Logistic Regression (0.818) and then the Decision Tree (0.735). Ranked by recall on the minority class, the Neural Network again leads slightly (0.74) with the other two tied at 0.71. Taken together, Logistic Regression and the Neural Network are the stronger models for this task, with the Decision Tree remaining a reasonable but comparatively weaker baseline.

Conclusion

All three models now identify claims at a meaningfully useful rate: roughly 71 to 74% of actual claims are caught by each model, a dramatic improvement over an earlier version of this pipeline in which Logistic Regression and the Neural Network predicted “No” for essentially every record and the Decision Tree's ranking ability was statistically indistinguishable from chance. This outcome reinforces the report's central methodological point: on a severely imbalanced target, accuracy is not a reliable metric, and precision, recall, and especially AUC are what actually distinguish a useful model from a useless one. It also demonstrates that correcting how imbalance is handled, consistently, across every model, rather than for only one of them, can matter more to the final results than the choice of algorithm itself.

Model Performance Summary

- Decision Tree: regularized (max_depth=10, min_samples_leaf=20) and class-weighted; recall 0.71 on the minority class, AUC 0.735, a large improvement over an unconstrained tree that overfit the training data and scored AUC 0.541.
- Logistic Regression: class-weighted; recall 0.71, AUC 0.818, the best accuracy of the three, and no longer degenerates into predicting only the majority class.

- Neural Network: class-weighted with early stopping on validation AUC; recall 0.74, AUC 0.824, the strongest ranking performance of the three models.

Why the Corrected Models Perform Better

The original pipeline treated class imbalance inconsistently: only the Decision Tree received any imbalance handling, so Logistic Regression and the Neural Network had no incentive to ever predict the rare class, and their apparently “high” accuracy was simply a reflection of the 98.5%-majority baseline. Applying class weighting uniformly across all three models, and giving each model a fair, equivalent treatment of the imbalance, is what allowed Logistic Regression and the Neural Network to start identifying actual claims. Separately, the unconstrained Decision Tree's original AUC of 0.541 was not a sign that tree-based models are unsuited to this problem; it was a sign of severe overfitting from an unregularized tree with no depth or leaf-size limit. Constraining tree complexity resolved this without changing anything else about the approach.

Challenges Faced

- Handling imbalanced data was the central challenge throughout: even with stratified splitting, models needed explicit class weighting (not just a good split) to avoid defaulting to the majority class.
- Choosing appropriate evaluation metrics (precision, recall, and AUC rather than accuracy) was essential to correctly interpret results that would otherwise have looked deceptively strong.
- High-cardinality categorical features (Destination in particular, with 149 unique values and many singleton categories) increased dimensionality substantially and required care to encode without leaking test-set category information into training.
- Balancing Decision Tree complexity against overfitting required deliberate regularization; the default, unconstrained tree looked reasonable on paper (93.7% accuracy) while performing barely better than random guessing underneath.

Future Improvements

- Try resampling techniques such as SMOTE or majority-class undersampling as an alternative or complement to class weighting.
- Explore ensemble methods (Random Forest, Gradient Boosting) which may capture subtler interactions than a single Decision Tree.
- Tune the classification threshold explicitly (rather than the default 0.5) using a precision-recall curve, since business costs of false negatives and false positives are unlikely to be symmetric for an insurer.
- Perform feature engineering, for example aggregating rare Destination categories into broader regions, to reduce dimensionality and noise from the high-cardinality fields.
- Run a systematic hyperparameter search (e.g., grid or Bayesian search over tree depth, regularization strength, and network architecture) rather than the single fixed configuration used here.

In conclusion, correcting how missing data, encoding, splitting, and class imbalance were handled changed this project's results more than any change of algorithm would have: it moved all three models from effectively unusable on the minority class to consistently identifying 71 to 74% of real claims, while keeping the original three-model comparison structure intact.